

LABORATORIO 1

Ejercicio 1: Empleados ordenados

Código Java y sección del servidor

Tema	Arreglos ordenados por nombre de empleado
Operaciones	Alta, baja, modificación, búsqueda y listado
Archivos	N_Empleado.java, Empleado.java y Servidor.java

Enunciado

Una compañía necesita almacenar en arreglos la siguiente información de cada uno de sus empleados ordenados alfabéticamente por el nombre: nombre, dirección, edad, sexo y años de antigüedad.

El programa permite listar los empleados, dar de alta, dar de baja, modificar los años de antigüedad, listar un empleado determinado y salir.

Estructura de archivos

- N_Empleado.java: clase que almacena los datos de un empleado.
- Empleado.java: clase principal con el arreglo ordenado y las operaciones.
- Servidor.java: servidor HTTP que conecta la página HTML con la lógica Java.

1. Clase N_Empleado.java

Crea esta clase en la misma carpeta o paquete del proyecto Java.

```
public class N_Empleado {
    private String nombre;
    private String direccion;
    private int edad;
    private String sexo;
    private int antigüedad;

    public N_Empleado(String nombre, String direccion, int edad, String sexo, int antigüedad) {
        this.nombre = nombre;
        this.direccion = direccion;
        this.edad = edad;
        this.sexo = sexo;
        this.antigüedad = antigüedad;
    }

    public String getNombre() {
        return nombre;
    }

    public String getDireccion() {
        return direccion;
    }
}
```

```
public int getEdad() {
    return edad;
}

public String getSexo() {
    return sexo;
}

public int getAntiguedad() {
    return antiguedad;
}

public void setAntiguedad(int antiguedad) {
    this.antiguedad = antiguedad;
}

@Override
public String toString() {
    return nombre + " | " + direccion + " | " + edad + " | " + sexo + " | " + antiguedad;
}
}
```

2. Clase Empleado.java

Esta es la clase principal. El arreglo se mantiene ordenado alfabéticamente por el nombre cada vez que se registra un empleado.

```
import java.util.Arrays;

public class Empleado {
    private N_Empleado[] empleados;
    private int tam;

    public Empleado() {
        empleados = new N_Empleado[100];
        tam = 0;
    }

    private int buscarPosicion(String nombre) {
        int inicio = 0;
        int fin = tam - 1;

        while (inicio <= fin) {
            int medio = (inicio + fin) / 2;
            int comparacion = empleados[medio].getNombre().compareToIgnoreCase(nombre);

            if (comparacion == 0) {
                return medio;
            }

            if (comparacion < 0) {
                inicio = medio + 1;
            } else {
                fin = medio - 1;
            }
        }

        return -1;
    }

    private int posicionInsertar(String nombre) {
        int i = 0;

        while (i < tam && empleados[i].getNombre().compareToIgnoreCase(nombre) < 0) {
            i++;
        }

        return i;
    }

    public String alta(String nombre, String direccion, int edad, String sexo, int antiguedad) {
```

```
        if (tam == empleados.length) {
            return "No hay espacio para más empleados";
        }

        if (buscarPosicion(nombre) != -1) {
            return "El empleado ya existe";
        }

        int pos = posicionInsertar(nombre);

        for (int i = tam; i > pos; i--) {
            empleados[i] = empleados[i - 1];
        }

        empleados[pos] = new N_Empleado(nombre, direccion, edad, sexo, antiguedad);
        tam++;
        return "Empleado agregado correctamente";
    }

    public String baja(String nombre) {
        int pos = buscarPosicion(nombre);

        if (pos == -1) {
            return "El empleado no existe";
        }

        for (int i = pos; i < tam - 1; i++) {
            empleados[i] = empleados[i + 1];
        }

        empleados[tam - 1] = null;
        tam--;
        return "Empleado eliminado correctamente";
    }

    public String modificarAntiguedad(String nombre, int antiguedad) {
        int pos = buscarPosicion(nombre);

        if (pos == -1) {
            return "El empleado no existe";
        }

        empleados[pos].setAntiguedad(antiguedad);
        return "Antigüedad modificada correctamente";
    }

    public String listarUno(String nombre) {
        int pos = buscarPosicion(nombre);

        if (pos == -1) {
            return "El empleado no existe";
        }

        return empleados[pos].toString();
    }

    public String listarTodos() {
        if (tam == 0) {
            return "No hay empleados registrados";
        }

        StringBuilder respuesta = new StringBuilder();

        for (int i = 0; i < tam; i++) {
            respuesta.append(empleados[i].toString()).append("\n");
        }

        return respuesta.toString();
    }

    public N_Empleado[] getEmpleados() {
        return Arrays.copyOf(empleados, tam);
    }
}
```

3. Servidor.java

Si ya tienes un servidor para los ejercicios desordenados, agrega esta sección al mismo archivo. Si vas a probar solamente este ejercicio, puedes usar el servidor completo siguiente.

```
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;
import java.util.HashMap;
import java.util.Map;

public class Servidor {
    private static Empleado empleados = new Empleado();

    public static void main(String[] args) throws IOException {
        HttpServer servidor = HttpServer.create(new InetSocketAddress(8080), 0);
        servidor.createContext("/ordenado1/alta", new AltaHandler());
        servidor.createContext("/ordenado1/baja", new BajaHandler());
        servidor.createContext("/ordenado1/modificar", new ModificarHandler());
        servidor.createContext("/ordenado1/listarUno", new ListarUnoHandler());
        servidor.createContext("/ordenado1/listarTodos", new ListarTodosHandler());
        servidor.setExecutor(null);
        servidor.start();
        System.out.println("Servidor iniciado en el puerto 8080");
    }

    private static void cors(HttpExchange intercambio) {
        intercambio.getResponseHeaders().set("Access-Control-Allow-Origin", "*");
        intercambio.getResponseHeaders().set("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
        intercambio.getResponseHeaders().set("Access-Control-Allow-Headers", "Content-Type");
    }

    private static Map<String, String> leerDatos(HttpExchange intercambio) throws IOException {
        String datos = new String(intercambio.getRequestBody().readAllBytes(),
            StandardCharsets.UTF_8);
        Map<String, String> mapa = new HashMap<>();

        for (String parte : datos.split("&")) {
            String[] dato = parte.split("=", 2);
            if (dato.length == 2) {
                String llave = URLDecoder.decode(dato[0], StandardCharsets.UTF_8);
                String valor = URLDecoder.decode(dato[1], StandardCharsets.UTF_8);
                mapa.put(llave, valor);
            }
        }

        return mapa;
    }

    private static void responder(HttpExchange intercambio, String respuesta) throws IOException {
        cors(intercambio);
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            intercambio.sendResponseHeaders(204, -1);
            return;
        }

        byte[] bytes = respuesta.getBytes(StandardCharsets.UTF_8);
        intercambio.getResponseHeaders().set("Content-Type", "text/plain; charset=UTF-8");
        intercambio.sendResponseHeaders(200, bytes.length);

        try (OutputStream salida = intercambio.getResponseBody()) {
            salida.write(bytes);
        }
    }

    static class AltaHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange intercambio) throws IOException {
```

```
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            responder(intercambio, "");
            return;
        }

        Map<String, String> datos = leerDatos(intercambio);
        String respuesta = empleados.alta(
            datos.get("nombre"),
            datos.get("direccion"),
            Integer.parseInt(datos.get("edad")),
            datos.get("sexo"),
            Integer.parseInt(datos.get("antiguedad"))
        );
        responder(intercambio, respuesta);
    }
}

static class BajaHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange intercambio) throws IOException {
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            responder(intercambio, "");
            return;
        }

        Map<String, String> datos = leerDatos(intercambio);
        responder(intercambio, empleados.baja(datos.get("nombre")));
    }
}

static class ModificarHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange intercambio) throws IOException {
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            responder(intercambio, "");
            return;
        }

        Map<String, String> datos = leerDatos(intercambio);
        String respuesta = empleados.modificarAntiguedad(
            datos.get("nombre"),
            Integer.parseInt(datos.get("antiguedad"))
        );
        responder(intercambio, respuesta);
    }
}

static class ListarUnoHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange intercambio) throws IOException {
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            responder(intercambio, "");
            return;
        }

        Map<String, String> datos = leerDatos(intercambio);
        responder(intercambio, empleados.listarUno(datos.get("nombre")));
    }
}

static class ListarTodosHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange intercambio) throws IOException {
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            responder(intercambio, "");
            return;
        }

        responder(intercambio, empleados.listarTodos());
    }
}
}
```

Rutas que utilizará la página HTML

La sección JavaScript del ejercicio debe enviar los datos con método POST a estas rutas:

Operación	Ruta del servidor
Dar de alta	/ordenado1/alta
Dar de baja	/ordenado1/baja
Modificar antigüedad	/ordenado1/modificar
Listar un empleado	/ordenado1/listarUno
Listar todos	/ordenado1/listarTodos

Importante para integrarlo

- Los nombres de los campos enviados desde HTML deben ser: nombre, direccion, edad, sexo y antigüedad.
- El arreglo no usa base de datos: los datos existen mientras el servidor Java está ejecutándose.
- La inserción mueve los elementos necesarios para conservar el orden alfabético por nombre.
- La búsqueda de un empleado se realiza con búsqueda binaria porque el arreglo permanece ordenado.



ARREGLOS ORDENADOS

Ejercicio 1

Empleados ordenados por nombre



Abrir ejercicio



Documento

Arreglos Ordenados: Ejercicio 1

Menú de Opciones



Modo Oscuro

Configurar arreglo

Tamaño del arreglo:

Ejemplo: 5

Iniciar ejercicio

Salir

Arreglos Ordenados: Ejercicio 1

Menú de Opciones

🌙 Modo Oscuro

1. Listar todos

2. Dar de alta

3. Dar de baja

4. Modificar años de antigüedad

5. Listar un empleado

6. Salir



Ejercicio iniciado

Arreglo creado correctamente con capacidad para 2 empleados

OK



Arreglos Ordenados: Ejercicio 1

Menú de Opciones

🌙 Modo Oscuro

1. Listar todos

2. Dar de alta

3. Dar de baja

4. Modificar años de antigüedad

5. Listar un empleado

6. Salir

Lista de Empleados Registrados

← Regresar

🌙 Modo Oscuro

No hay empleados registrados



Arreglos Ordenados: Ejercicio 1

Menú de Opciones

 Modo Oscuro

1. Listar todos

2. Dar de alta

3. Dar de baja

4. Modificar años de antigüedad

5. Listar un empleado

6. Salir

Dar de alta

Nombre completo:

Dirección:

Edad:

Sexo:



Años de antigüedad:

Guardar



Arreglos Ordenados: Ejercicio 1

Menú de Opciones

 Modo Oscuro

1. Listar todos

2. Dar de alta

3. Dar de baja

4. Modificar años de antigüedad

5. Listar un empleado

6. Salir

Dar de baja

Nombre del empleado:

Eliminar

Arreglos Ordenados: Ejercicio 1

Menú de Opciones

 Modo Oscuro

1. Listar todos

2. Dar de alta

3. Dar de baja

4. Modificar años de antigüedad

5. Listar un empleado

6. Salir

Modificar años de antigüedad

Nombre del empleado:

Nuevos años de antigüedad:

Menú de Opciones

 Modo Oscuro

1. Listar todos

2. Dar de alta

3. Dar de baja

4. Modificar años de antigüedad

5. Listar un empleado

6. Salir

Listar un empleado

Nombre del empleado: